

LiteTest Documentation

Guide

Document Version: 1.0

Runner Version: 1.0.0

Test Version: 1.0.0

This document is a guide to the LiteTest documentation.

Copyright (c) 2026 Paul Sinclair

License

SPDX-License-Identifier: MIT

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Preface

This document is intended for LiteTest users and contributors who need quick access to the definitions of terms used in LiteTest or to browse through the terms.

For a glossary of terms, see the LiteTest Glossary Reference (`LiteTest_Glossary_Reference.md`).

Document Version History

Document	Runner	Test	Date	Comment	Author/Editor
1.0	1.0.0	1.0.0	2026-06-11	Initial version.	Paul Sinclair

- The **Document** column records the document's version with the format `M.u` (Major, update).
- The **Runner** column records the LiteTest Runner API version current at the time this document version was published and is defined by its `LT_RUNNER_VERSION` macro.
- The **Test** column records the LiteTest Test API version current at the time this document version was published and is defined by its `LT_TEST_VERSION` macro.
- Both Runner and Test use the version format "`M.m.p`" (Major, minor, patch).
- `M` is the same for the Document, Runner, and Test versions.

The document's update version tracks updates to this document and does not correspond to a minor or patch version. `u` increments whenever this document is updated without a change to `M`, and it resets to 0 when `M` is incremented.

Table of Contents

1. Introduction . . . 3
2. LiteTest Documents . . . 4
3. LiteTest Repository Layout . . . 5

1. Introduction

The LiteTest documentation consists of documents for:

- **Users:** Public-facing framework and API documents,
- **Contributors;** Guide and API internal documents.

Document types:

- **README:** Introduction or description.
- **Guide:** Usage information.
- **Reference:** Reference companion to a guide.

2. LiteTest Documents

The following are the LiteTest user and contributor documents.

User documentation:

- **README.md** — Introduction to LiteTest.
- **LiteTest_Documentation_Guide.md**: A guide to LiteTest documents and the repository layout.
- **LiteTest_Glossary_Reference.Md**: An alphabetically ordered list of terms generally used in LiteTest (emphasizing their specific meaning in LiteTest) and terms often used in the testing domain.
- **LiteTest_Runner_User_Guide.md**: Concepts, usage, and examples for the LiteTest Runner framework and API.
- **LiteTest_Runner_Reference.md**: Reference document for the LiteTest Runner API.
- **LiteTest_Test_User_Guide.md**: Concepts, usage, and examples for the LiteTest Test API.
- **LiteTest_Test_Reference.md**: Reference document for the LiteTest Test API.

Contributor documentation:

- **README.md**: Directory description, one for each subdirectory.
- **LiteTest_Contributor_Guide.md**: Versioning, documentation/coding guidelines, branching, testing, and CI/release checklists.
- **LiteTest_Runner_Internal_Guide.md**: Implementation concepts, architecture, and high-level design for the Runner API.
- **LiteTest_Runner_Internal_Reference.md**: Reference for the implementation of the Runner API.
- **LiteTest_Test_Internal_Guide.md**: Implementation concepts, architecture, and high-level design for the Test API.
- **LiteTest_Test_Internal_Reference.md**: Reference for the implementation of the Test API.

3. LiteTest Repository Layout

The following shows the layout of the GitHub repository paulsinclair51/LiteTest (core files and directories):

```
.github/workflows/ci.yml
README.md
LICENSE
.gitignore
Makefile
build_test_litetest.ps1
build/                # Build outputs and related artifacts.
docs/                 # User and contributor documentation.
    LiteTest_Documentation_Guide.md
    LiteTest_Glossary_Reference.md
    LiteTest_Runner_User_Guide.md
    LiteTest_Runner_Reference.md
    LiteTest_Contributor_Guide.md
    LiteTest_Runner_Internal_Guide.md
    LiteTest_Runner_Internal_Reference.md
    LiteTest_Test_Internal_Guide.md
    LiteTest_Test_Internal_Reference.md
examples/             # Usage examples.
include/              # Public API headers.
    litetest_runner.h
    litetest_test.h
reports/              # Generated report files (non-source).
scripts/              # Automation scripts.
    test/
src/                  # API source.
    litetest_runner.c
    litetest_test.c
tests/                # Testing assets.
    src/              # Test source files.
        test_litetest.c
        test_orchestrator.c
        test_file_compare_helpers.c
        test_guard1.c
        test_guard2.c
    include/           # Test headers.
    control/           # Golden/baseline files for comparison.
output/               # Retained test outputs needing review
                        # (e.g., a copied report file to compare
                        # against control/)
                        # (mismatch/new); matched outputs
                        # removed by default (use -o to keep).
input/                # Static test input files.
```

```
tmp/          # Temporary test outputs; removed at run
               # start for clean start; auto-removed by
               # test executable when it finishes (use -t
               # to keep for debugging); user can
               # manually remove as needed.
```